SPI Interface Project

Digital design using Verilog and FPGA flow using Vivado

Zeyad Saber Hussein

Ziad Mohamed Sobhy

Youssef Hossam Elzainy

Table of Contents

1 Project Overview 3

1.1 Description 3

1.2 SPI Slave with Single Port RAM Schematic 3

1.3 Ports explanation 4

1.4 Read/Write Command explanation 4

1.5 Finite State Machine – F`SM 5

2 RTL Code 6

2.1 Single Port Synchronous RAM module 6

2.2 SPI module 7

2.3 SPI Wrapper module 10

3 SPI Testbench module 11

4 Do File 13

5 Questa sim Waveform 13

6 Constraint file 14

7 One hot encoding 15

7.1 Elaboration schematic 15

7.2 Elaboration messages 15

7.3 Synthesis schematic 16

7.4 Synthesis messages 16

7.5 Implementation device 17

7.6 Timing report 17

8 Seq encoding 18

8.1 Elaborated schematic 18

8.2 Synthesis showing critical path 18

8.3 Implementation 18

9 Gray encoding 19

9.1 Elaboration schematic 19

9.2 Synthesis schematic showing critical path 20

9.3 Implementation 20

9.4 Bitstream Generation 21

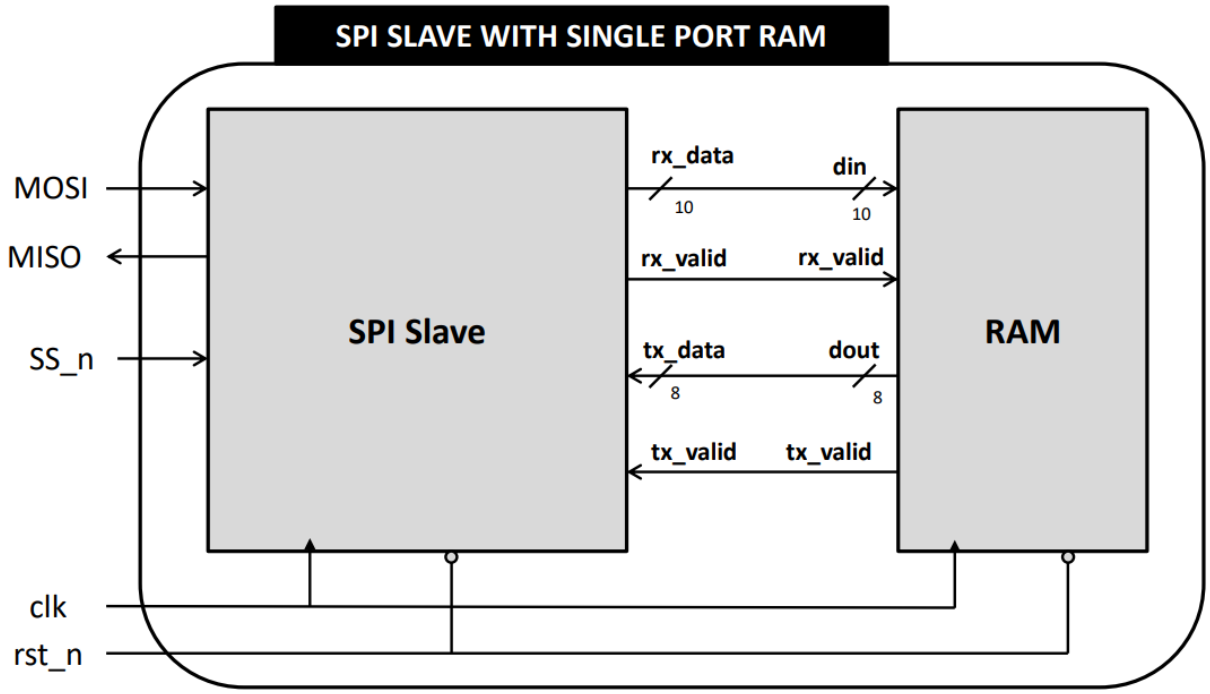
10 Lint check 21

1 Project Overview

1.1 Description

This project implements an SPI-based communication system integrated with a synchronous RAM module. The SPI interface receives serial data, converts it into parallel form, and passes it to the RAM for storage or retrieval based on control signals. The system operates synchronously using a clock signal and uses handshake signals to indicate valid data transfer between modules.

1.2 SPI Slave with Single Port RAM Schematic



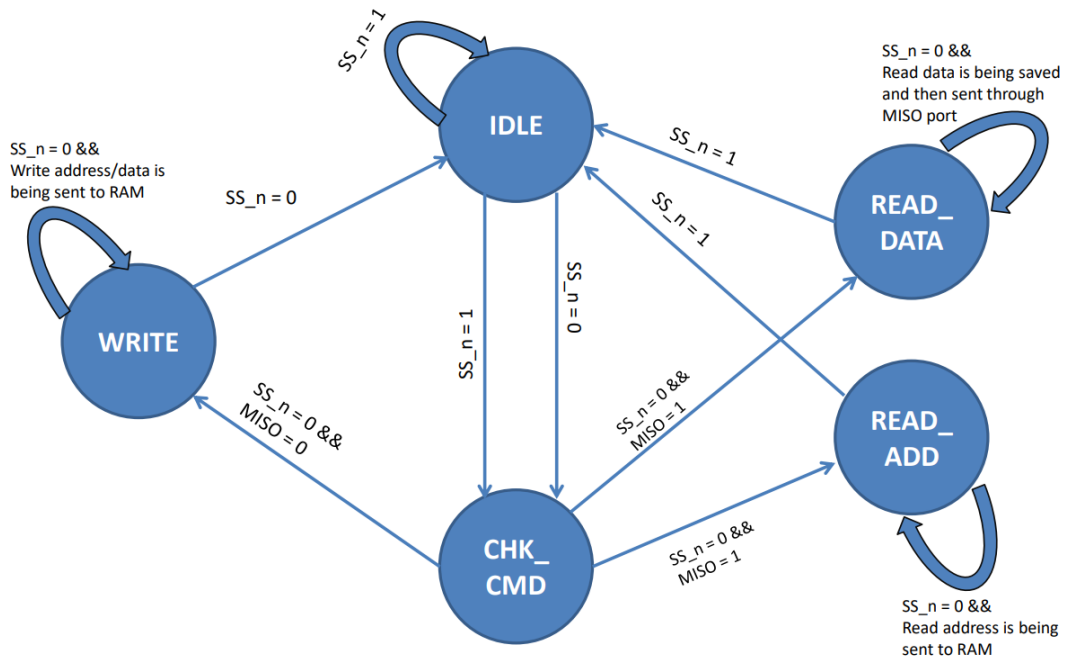
1.3 Ports explanation

Port Name	Direction	Width	Description
Din	Input	10-bit	Input data from SPI interface
clk	Input	1-bit	System clock
rst	Input	1-bit	Reset signal
rx_valid	Input	1-bit	Indicates valid received data
Dout	Output	8-bit	Output data from memory
tx_valid	Output	1-bit	Indicates valid output data

1.4 Read/Write Command explanation

Port	Din[9:8]	Operation
Hold Write Address	00	Hold din[7:0] internally as write address
Write Data	01	Write din[7:0] in the memory with write address held previously
Hold Read Address	10	Hold din[7:0] internally as read address
Read Data	11	Read the memory with read address held previously, tx_valid should be HIGH, dout holds the word read from the memory, ignore din[7:0]

1.5 Finite State Machine – F`SM



2 RTL Code

2.1 Single Port Synchronous RAM module

```
module Sync_Ram #(
    parameter mem_width = 8,
    parameter mem_depth = 256,
    parameter addr_size = 8
) (
    input [9 : 0] Din,
    input clk,
    input rst,
    input rx_valid,
    output reg [7 : 0] Dout,
    output reg tx_valid
);
    reg [mem_width - 1 : 0] mem [mem_depth - 1 : 0];
    reg [7 : 0] write_address, read_address;

    always @(posedge clk) begin
        if (~rst) begin
            Dout <= 0;
            tx_valid <= 0;
            read_address <= 0;
            write_address <= 0;
        end else if (rx_valid) begin
            case (Din[9:8])
                2'b00: write_address <= Din [7 : 0];
                2'b01: mem [write_address] <= Din[7 : 0];
                2'b10: read_address <= Din[7 : 0];
                2'b11: Dout <= mem [read_address];
                default: Dout <= mem[read_address];
            endcase
        end
        tx_valid <= (Din[9] && Din[8] && rx_valid)? 1 : 0;
    end

endmodule
```

2.2 SPI module

```
module Spi_slave #(
    parameter IDLE = 3'b000,
    parameter WRITE =3'b001,
    parameter CHK_CMD =3'b010,
    parameter READ_ADD =3'b011,
    parameter READ_DATA =3'b100
) (
    input MOSI,ss_n,clk,rst,tx_valid, input [7:0] tx_data,
    output reg[9:0] rx_data, output reg rx_valid, MISO
);
    reg get_add; // internal signal to identify to either read address or
read data (0 -> read address , 1 -> read data)
    reg [2:0] cs,ns ;
    reg [3:0] IO_Ctr;


// NEXT STATE MODULE

   always @(ss_n,MOSI) begin

        if (ss_n ) begin
            ns <= IDLE;
            if (cs==READ_ADD)   get_add =1; // read address and set
get_add
            else if (cs==READ_DATA)     get_add =0; // read data and clear
ger_add
        end

        else begin
        case (cs)
           IDLE : ns <= CHK_CMD;
           WRITE : ns<= WRITE;
           READ_ADD : ns<= READ_ADD;
           READ_DATA : ns<= READ_DATA;
           CHK_CMD : if (get_add && MOSI) ns<=READ_DATA;
                    else if (MOSI) ns<= READ_ADD;
                    else ns<= WRITE;
            default: ns<=IDLE;
        endcase
        end
   end

// CURRENT STATE MODULE
```

```
always @(posedge clk ) begin
    if (~rst)
        cs <=IDLE;
    else
        cs <= ns;

end


// OUTPUT LOGIC MODULE

always @(posedge clk) begin
    if (~rst) begin
        get_add <= 0; // get address -> (0 -> read address, 1 -> read
data)
        rx_data <= 0;
        rx_valid <= 0;
        MISO <= 0;
    end
    else begin
        case (cs)
            IDLE: rx_valid <= 0; // wait mode

            CHK_CMD: IO_Ctr <= 10; // Setting the counter by 10 for the
serial read/write operation

            WRITE: begin
                if (IO_Ctr > 0) begin
                    rx_valid <= 0; // still reading data from the master
                    rx_data[IO_Ctr - 1] <= MOSI; // receiving data bit by
bit from the MOSI
                    IO_Ctr <= IO_Ctr - 1; // decrementing the counter by
one
                end
                else begin
                    rx_valid <= 1; // when the counter reaches the zero ->
this means that the data is received (rx_valid = 1)
                end
            end

            READ_ADD: begin
                if (IO_Ctr > 0) begin
                    rx_data[IO_Ctr - 1] <= MOSI; // reading the address
from the master
```

```
                    IO_Ctr <= IO_Ctr - 1; // decrementing the counter
after each step
                end
                else begin
                    rx_valid <= 1; // hen the counter reaches the zero ->
this means that the data is received (rx_valid = 1)
                end
            end

            READ_DATA: begin
                if (tx_valid) begin // if tx_valid is set -> means that
the tx_data is ready to be sent to the master
                    rx_valid <= 0;
                    if (IO_Ctr > 0) begin
                        MISO <= tx_data[IO_Ctr - 1]; // sending data to
the MISO bit by bit
                        IO_Ctr <= IO_Ctr - 1; // decrementing the counter
by one each step
                    end
                end
                else begin

                    if (IO_Ctr > 0) begin
                        rx_data[IO_Ctr - 1] <= MOSI;
                        IO_Ctr <= IO_Ctr - 1;
                    end

                    else begin
                        rx_valid <= 1; // the data is ready in the ram
                        IO_Ctr <= 8;
                    end
                end
            end
        endcase
    end
end

endmodule
```

2.3 SPI Wrapper module

```
module Spi_Wrapper (
    input clk , rst,MOSI,ss_n,
    output MISO
);
    wire [9:0] rx_data;
    wire rx_valid,tx_valid;
    wire [7:0] tx_data ;
    Spi_slave spi(.MOSI(MOSI), .clk(clk), .rst(rst), .ss_n(ss_n),
.MISO(MISO), .tx_data(tx_data),
    .rx_data(rx_data), .rx_valid(rx_valid), .tx_valid(tx_valid));

    Sync_Ram RAM(rx_data,clk,rst,rx_valid,tx_data,tx_valid);

endmodule
```

3 SPI Testbench module

```
module SPI_TB();

reg clk, rst_n, ss_n, mosi; // Input Ports in Design
wire miso; // Output Ports in Design
reg [9:0] test_data; // Test data that will applied on mosi
integer i = 0; // Variable used to iterate over loops

// Module Instantiation
Spi_Wrapper DUT (.clk(clk), .rst(rst_n), .MOSI(mosi), .ss_n(ss_n),
.MISO(miso));

// Clock Generation
initial begin
    clk = 0;
    forever begin
        #1;
        clk=~clk;
    end
end

// Any @(negedge clk); just to see the change in signals in simulation
initial begin
    // ------Initializing memory------
    $readmemh("mem.dat",DUT.RAM.mem);
    // ------Testing Reset------
    rst_n = 0; // Activating Reset
    ss_n  = 0;
    mosi  = 0;
    test_data = $random;
    @(negedge clk);
    rst_n = 1; // Deactivating Reset
    // ------Testing Read Address------
    ss_n = 0;
    @(negedge clk); //////??????????
    mosi = 1;
    //@(negedge clk); // Not sure correct or not ,  it will work with
which test_data??
    test_data = 10'b1001_1010;
    for(i=10;i>0;i=i-1) begin
        @(negedge clk);
        mosi = test_data[i-1];
    end
    ss_n = 1;
```

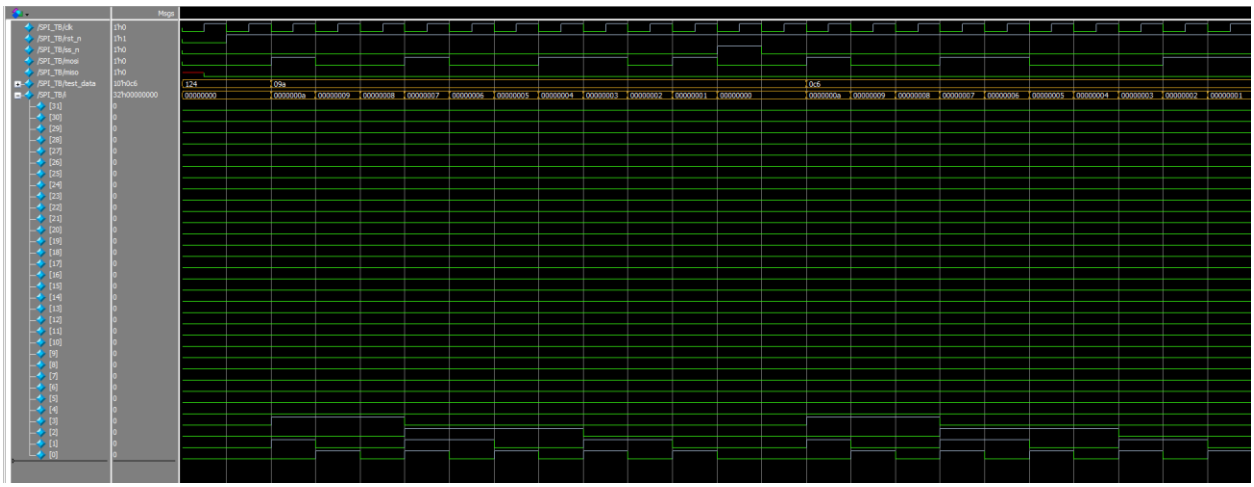
```
    @(negedge clk);
    ss_n = 0;
    @(negedge clk);
    mosi = 1;
    //@(negedge clk); // Not sure correct or not ,  it will work with
which test_data??
    test_data = 10'b1100_0110;
    for(i=10;i>0;i=i-1) begin
        @(negedge clk);
        mosi = test_data[i-1];
    end
    //@(negedge clk);
    // To See Output (moiso) we will introduce delay
    repeat(10) begin
        @(negedge clk);
    end
    ss_n = 1;
    @(negedge clk);
    ss_n = 0;
    @(negedge clk);
    mosi = 0;
    //@(negedge clk); // Not sure correct or not ,  it will work with
which test_data??
    test_data = 10'b0000_0110;
    for(i=10;i>0;i=i-1) begin
        @(negedge clk);
        mosi = test_data[i-1];
    end
    ss_n = 1;
    @(negedge clk);
    // ------Testing Write Data------
    ss_n = 0;
    @(negedge clk);
    mosi = 0;
    //@(negedge clk);
    test_data = 10'b0110_0100;
    for(i=10;i>0;i=i-1) begin
        @(negedge clk);
        mosi = test_data[i-1];
    end
    ss_n = 1;
    @(negedge clk);
    $stop;
end
endmodule
```

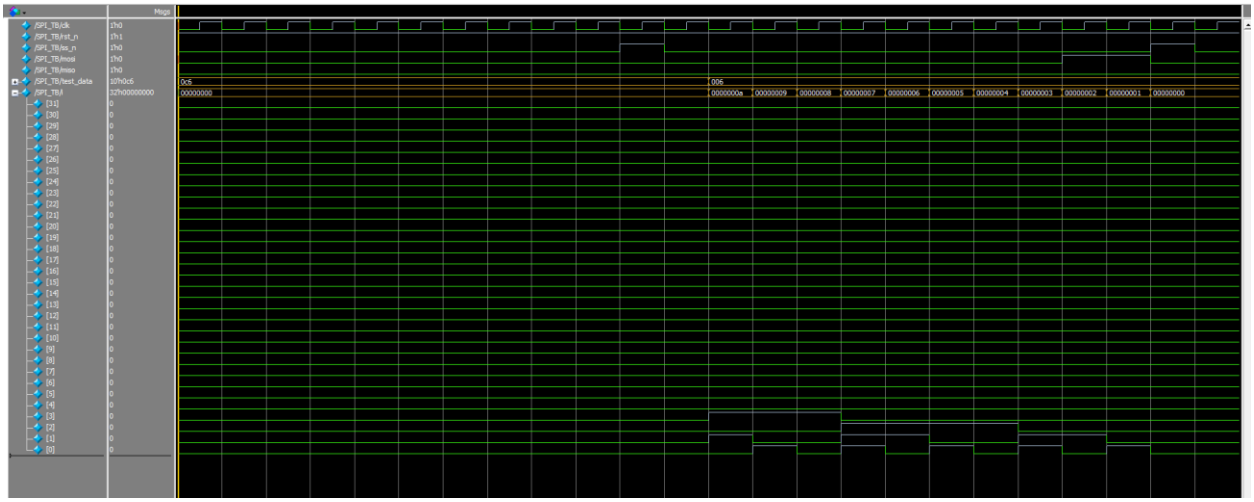
4 Do File

D: > digital_design_verilog > SPI_Interface_Project > run.do

```
vlib work
vlog Single_Port_Sync_RAM.v SPI.v SPI_SLAVE.v SPI_TB.v
vsim -voptargs=+acc work.SPI_TB
add wave *
run -all
```

5 Questa sim Waveform



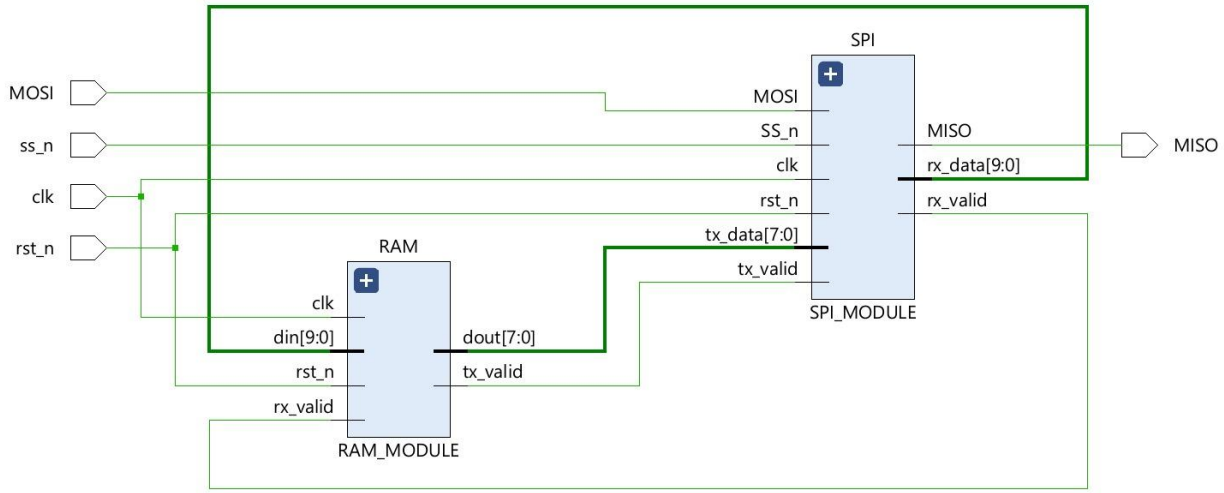


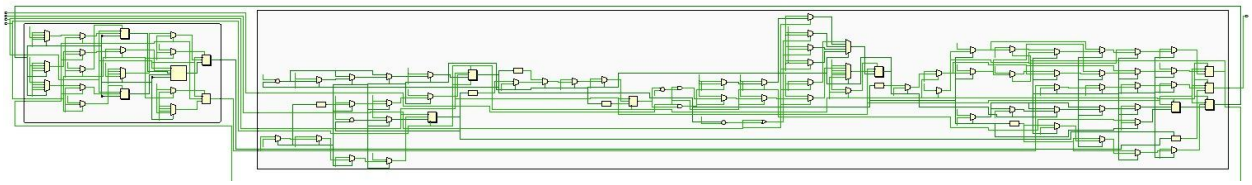
6 Constraint file

```
## This file is a general .xdc for the Basys3 rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to
the top level signal names in the project

## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports
clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports clk]


## Switches
set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports
{rst_n}]
set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports
{ss_n}]
set_property -dict { PACKAGE_PIN W16   IOSTANDARD LVCMOS33 } [get_ports
{MOSI}]

## LEDs
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports
{MISO}]

## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

## SPI configuration mode options for QSPI boot, can be used for all
designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

7 One hot encoding

7.1 Elaboration schematic





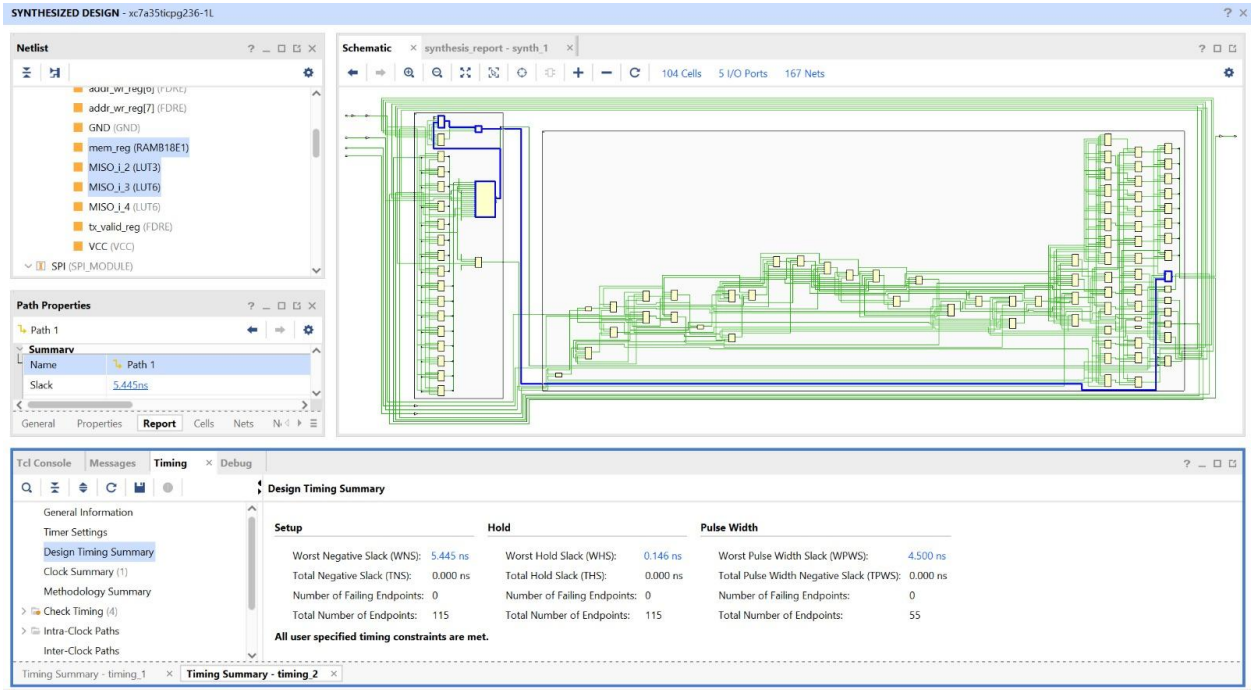
7.2 Elaboration messages

Elaborated Design (12 infos, 7 status messages)

General Messages (12 infos, 7 status messages)

- [Synth 8-7079] Multithreading enabled for synth_design using a maximum of 2 processes.
- [Synth 8-7078] Launching helper process for spawning children vivado processes
- [Synth 8-7075] Helper process launched with PID 11524
- [Synth 8-6157] synthesizing module 'SPI_Wrapper' [SPI_WRAPPER.v:1] (2 more like this)
- [Synth 8-155] case statement is not full and has no default [SPI.v:93]
- [Synth 8-6155] done synthesizing module 'SPI_MODULE' (0#1) [SPI.v:1] (2 more like this)
- [Project 1-570] Preparing netlist for logic optimization
- Processing XDC Constraints (6 more like this)
- [Project 1-111] Unisim Transformation Summary:
  No Unisim elements were transformed.

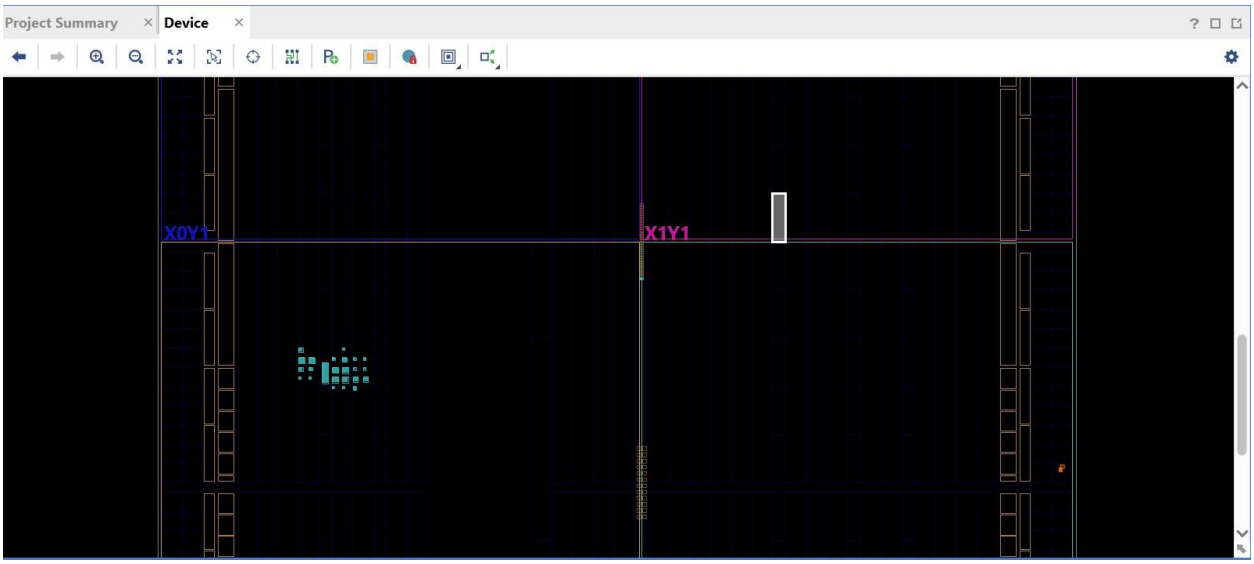
7.3 Synthesis schematic



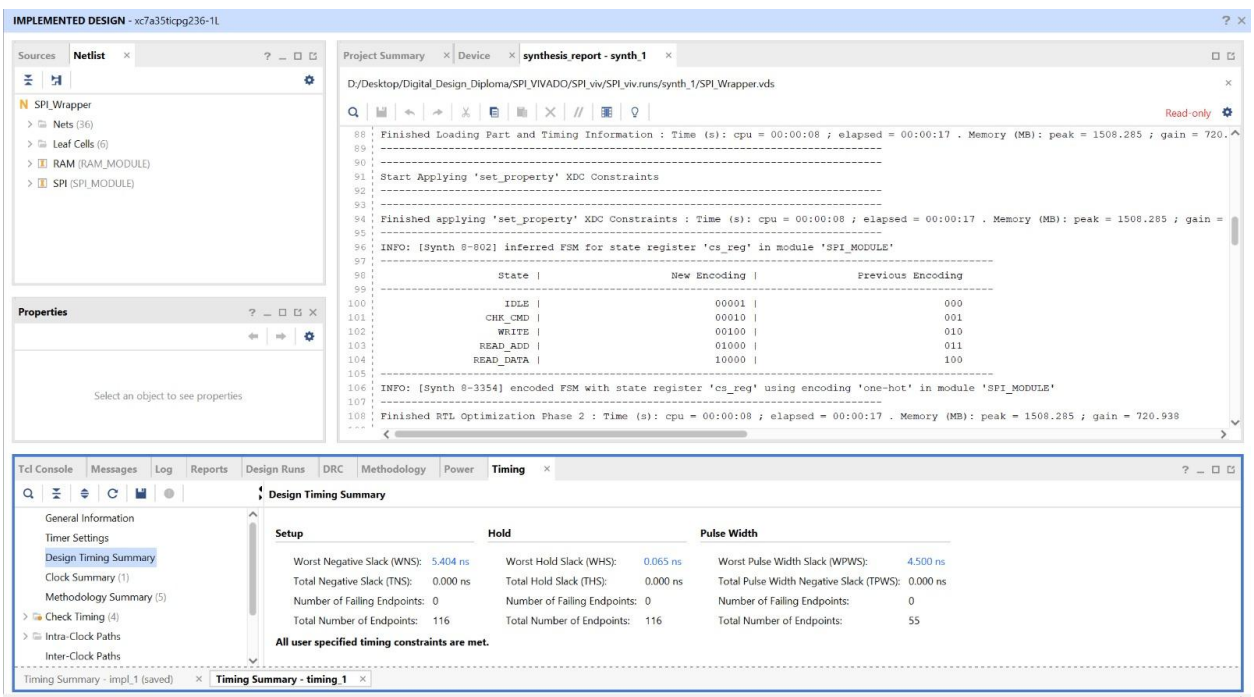
7.4 Synthesis messages

- Synthesized Design (12 infos, 2 status messages)
  - General Messages (12 infos, 2 status messages)
    - [Project 1-5699] Read binary netlist with skipMacroContent - 1
    - [Netlist 29-17] Analyzing 1 Unisim elements for replacement
    - [Netlist 29-28] Unisim Transformation completed in 0 CPU seconds
    - [Project 1-479] Netlist was created with Vivado 2025.2
    - [Project 1-570] Preparing netlist for logic optimization
    - Parsing XDC File [SPI.xdc] (1 more like this)
    - [Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).
    - [Project 1-111] Unisim Transformation Summary:
      No Unisim elements were transformed.
    - [Timing 38-35] Done setting XDC timing constraints.
    - [Timing 38-91] UpdateTimingParams: Speed grade: -1L, Delay Type: min_max. (1 more like this)
    - [Timing 38-191] Multithreading enabled for timing update using a maximum of 2 CPUs (1 more like this)

7.5 Implementation device

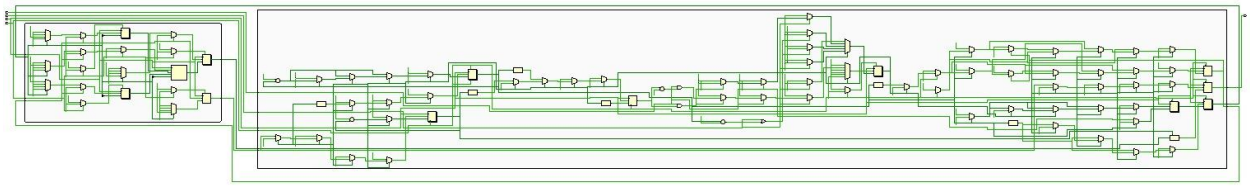


7.6 Timing report

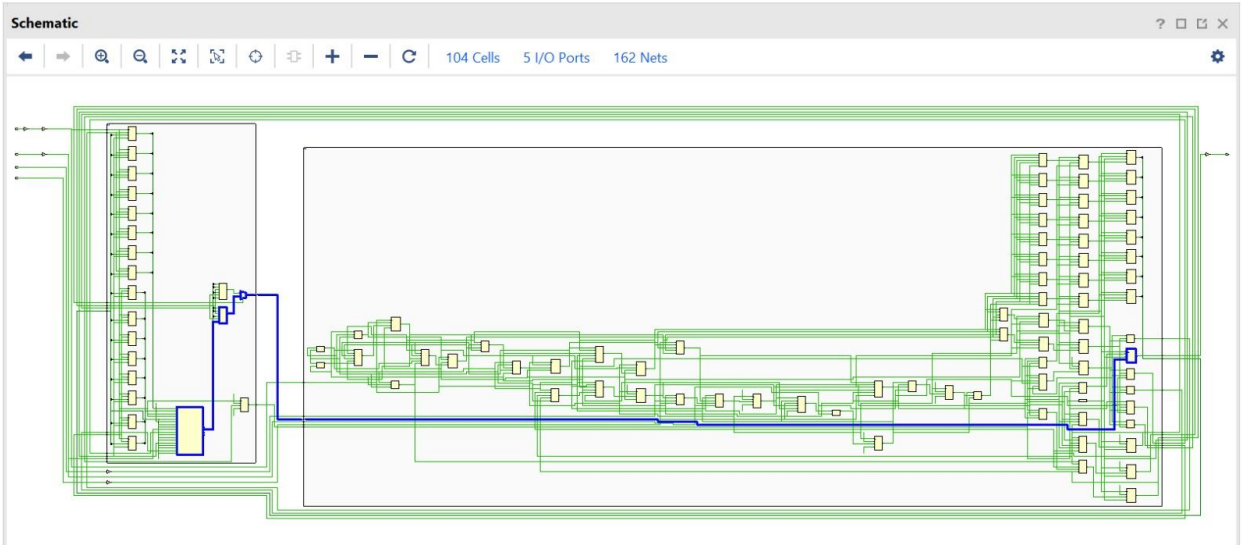


8 Seq encoding

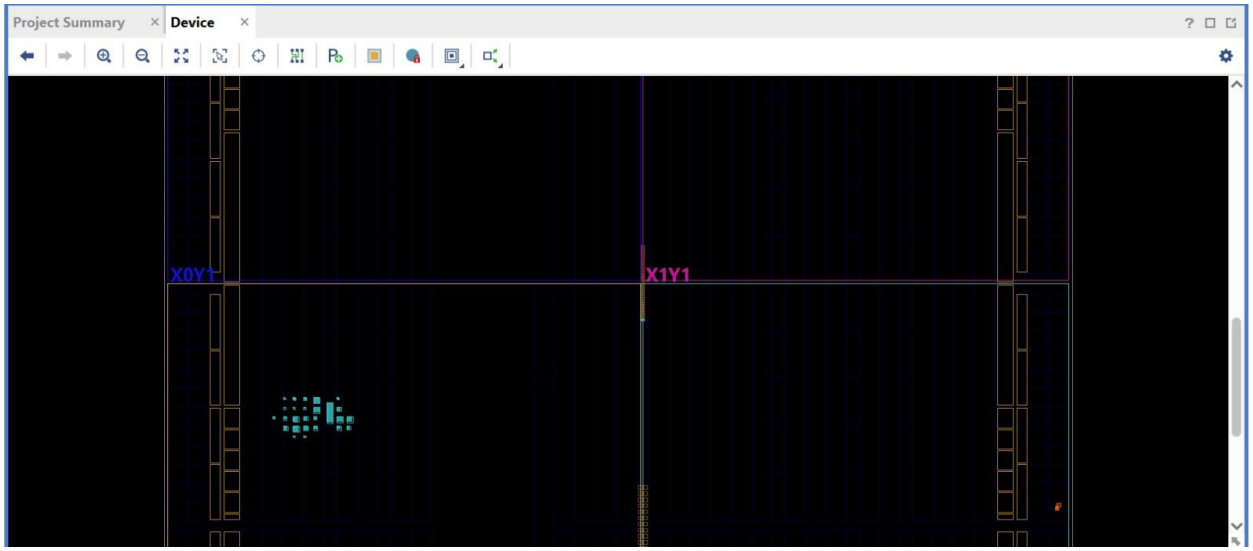
8.1 Elaborated schematic

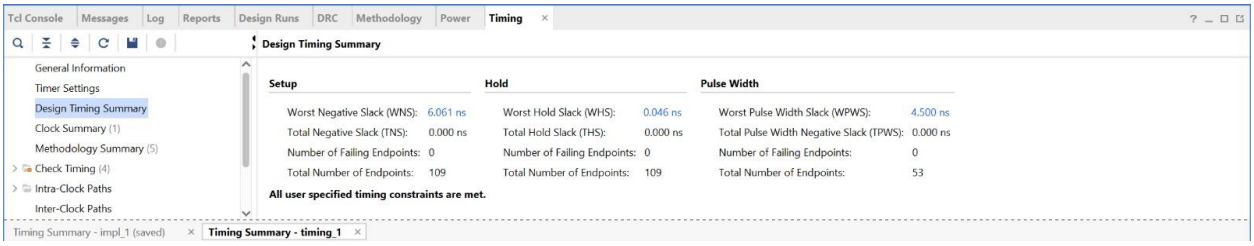


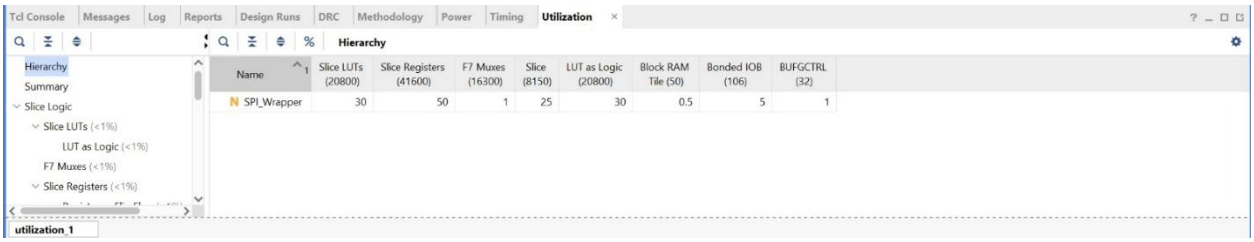
8.2 Synthesis showing critical path



8.3 Implementation

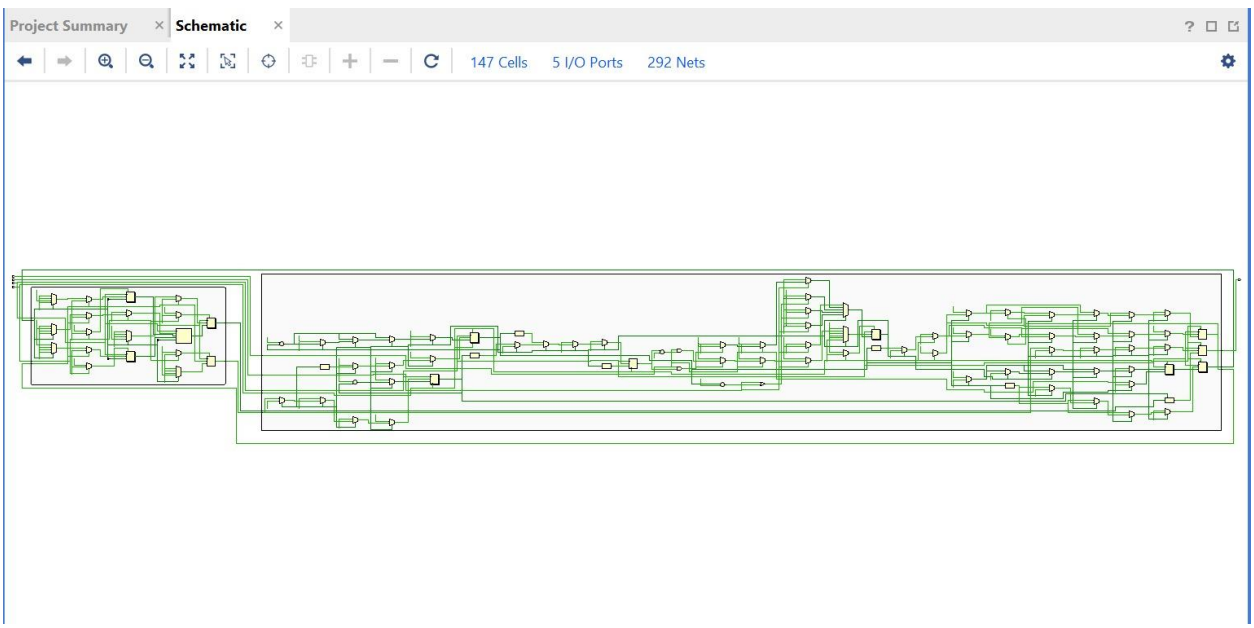




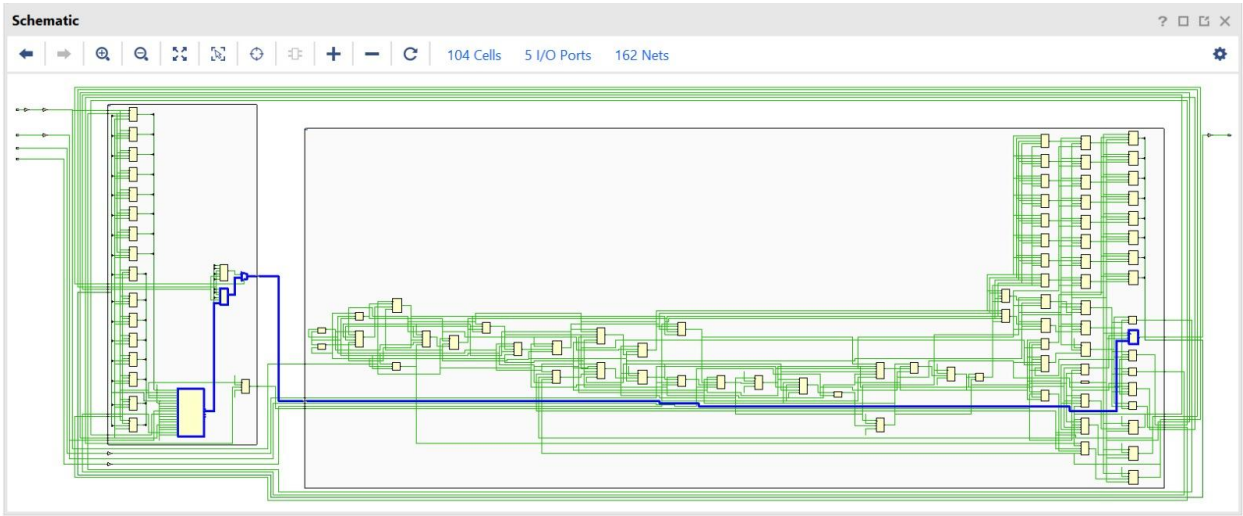


9 Gray encoding

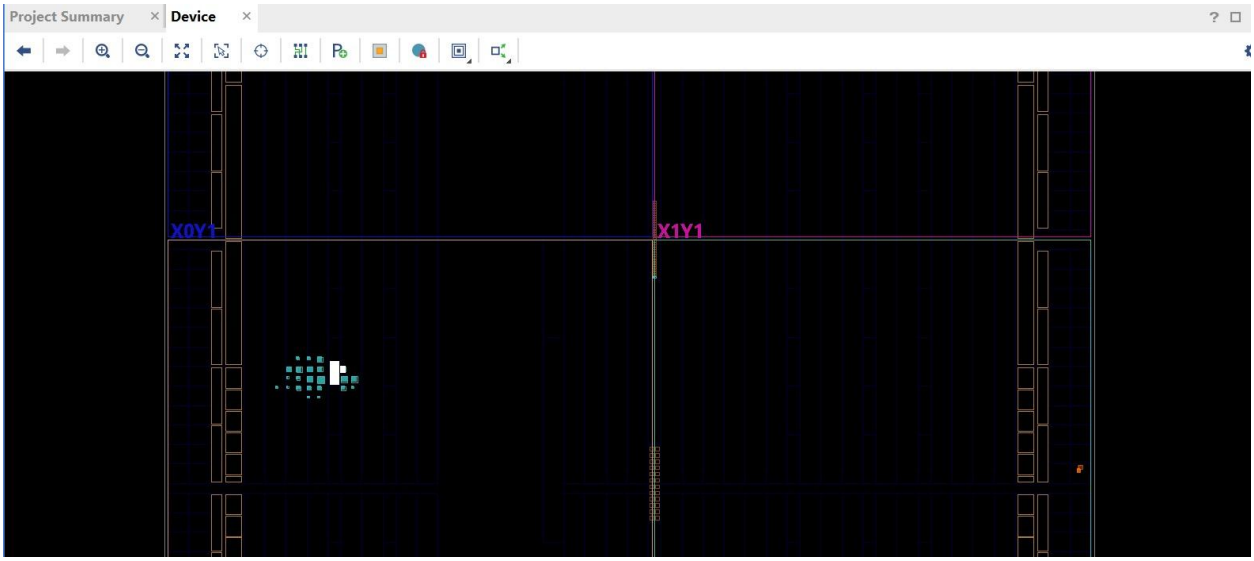
9.1 Elaboration schematic

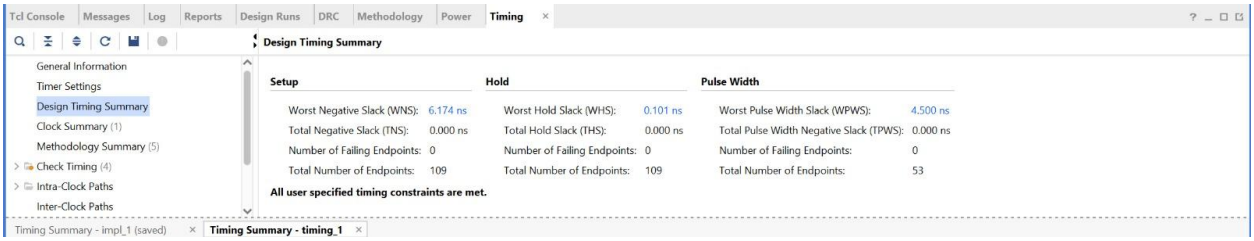


9.2 Synthesis schematic showing critical path

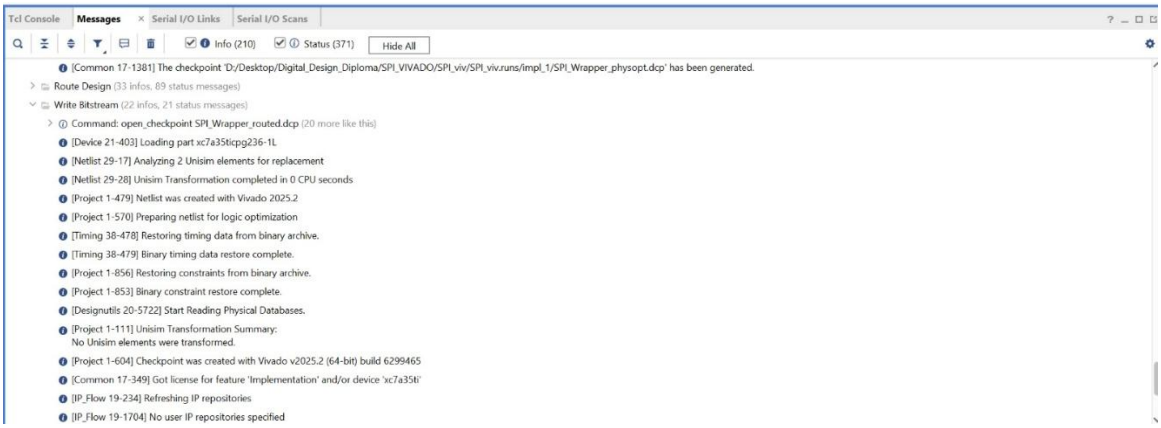


9.3 Implementation





9.4 Bitstream Generation



[IP_Flow 19-2313] Loaded Vivado IP repository 'D:/2025.2/Vivado/data/ip'.

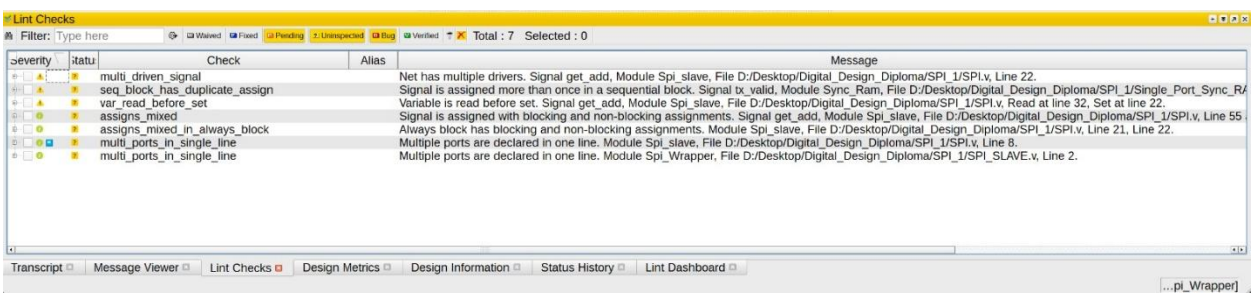
[DRC 23-27] Running DRC with 2 threads

[Vivado 12-3199] DRC finished with 0 Errors

[Vivado 12-3200] Please refer to the DRC report (report_drc) for more information.

[Designutils 20-2272] Running write_bitstream with 2 threads.

[Vivado 12-1842] Bitgen Completed Successfully.

[Common 17-83] Releasing license: Implementation

10 Lint check